

Лабораторная работа № 5 по курсу дискретного анализа: суффиксные деревья

Выполнил студент группы 08-207 МАИ *Дружинин Данила*.

Условие

1. Линеаризовать циклическую строку, то есть найти минимальный в лексикографическом смысле разрез циклической строки, с использованием суфф. дерева.
2. Вариант - 4. Е. 4 Линеаризация циклической строки

Метод решения

В задаче требуется линеаризовать циклическую строку, то есть выбрать такой её разрез, при котором полученная обычная строка будет лексикографически минимальной. Если дана строка s длины n , то все её возможные разрезы совпадают со всеми циклическими сдвигами этой строки. Следовательно, необходимо найти минимальный среди всех циклических сдвигов строки s .

Основная идея решения состоит в том, чтобы заменить работу с циклической строкой на работу с обычной строкой. Для этого исходная строка дублируется:

$$t = s + s.$$

В такой строке каждый циклический сдвиг исходной строки s является подстрокой длины n , начинающейся в одной из позиций от 0 до $n - 1$. Поэтому задача сводится к поиску лексикографически минимальной подстроки длины n среди подстрок строки $s + s$, соответствующих допустимым циклическим сдвигам.

Для эффективного поиска используется суффиксное дерево. Оно строится для строки

$$s + s + \{.$$

Символ $\{$ используется как специальный терминальный символ, обозначающий конец строки. Он не входит в исходную строку и лексикографически больше всех строчных латинских букв. Это важно для корректного выбора минимального пути в дереве: терминальный символ не должен быть выбран раньше обычных символов.

Суффиксное дерево представляет собой сжатый бор всех суффиксов строки. Каждый путь от корня до листа соответствует некоторому суффиксу строки. Рёбра дерева подписаны подстроками исходной строки. В программе эти подстроки не хранятся явно: каждая вершина хранит границы фрагмента строки, соответствующего ребру, которое ведёт в эту вершину. Такой подход позволяет не копировать подстроки и сохранять линейный объём памяти.

Построение суффиксного дерева выполняется алгоритмом Укконена. Алгоритм строит дерево онлайн, добавляя символы строки слева направо. На каждом шаге добавляется очередной символ, после чего дерево обновляется так, чтобы соответствовать уже обработанному префиксу.

Для ускорения построения используются следующие элементы:

- активная точка, задаваемая активной вершиной, активным ребром и активной длиной;
- суффиксные ссылки, позволяющие быстро переходить от одного суффикса к следующему;
- общий конец листовых рёбер, благодаря которому все листовые рёбра автоматически удлиняются при добавлении нового символа;
- разрезание ребра при появлении нового ветвления в дереве.

Активная точка показывает, в каком месте дерева продолжается построение. Она может находиться как в вершине, так и внутри ребра. Если при добавлении очередного символа нужного перехода из активной точки нет, создаётся новый лист. Если переход есть и следующий символ совпадает с добавляемым, значит соответствующий путь уже присутствует в дереве, и текущая фаза построения завершается. Если переход есть, но следующий символ отличается, ребро разрезается: создаётся новая внутренняя вершина, от которой отходят старое продолжение и новый лист.

После построения суффиксного дерева ответ находится лексикографическим обходом дерева. Начиная с корня, на каждом шаге выбирается минимальное исходящее ребро. Поскольку переходы в вершинах хранятся в массиве по индексам символов, минимальное ребро находится перебором символов от a до z , а затем терминального символа $\{$.

При выборе очередного ребра в ответ добавляются символы, соответствующие этому ребру. После этого переход продолжается из вершины, в которую это ребро ведёт. Обход заканчивается, когда длина ответа становится равной n , то есть длине исходной строки. Полученная строка является лексикографически минимальным циклическим сдвигом исходной строки.

Программа имеет следующую структуру:

- считывается исходная строка s ;
- строится строка $s + s + \{$;
- для этой строки строится суффиксное дерево;
- из корня дерева выполняется лексикографически минимальный проход;
- первые n символов этого прохода выводятся как ответ.

Корректность подхода основана на том, что все циклические сдвиги исходной строки являются подстроками длины n в строке $s + s$. Суффиксное дерево позволяет рассматривать эти подстроки через суффиксы строки $s + s + \{$. Лексикографически минимальный путь от корня дерева задаёт минимальное возможное продолжение строки, поэтому первые n символов этого пути дают минимальный разрез циклической строки.

Длина строки, для которой строится суффиксное дерево, равна $2n + 1$. Поскольку это линейно зависит от n , асимптотика построения остаётся линейной относительно длины исходной строки. Алгоритм Укконена строит суффиксное дерево за $O(n)$ времени при фиксированном алфавите. В программе алфавит состоит из 26 строчных латинских букв и одного терминального символа, поэтому переходы между вершинами выполняются за $O(1)$.

Поиск ответа также занимает $O(n)$ времени, так как в результирующую строку добавляется ровно n символов. Следовательно, общая временная сложность решения составляет

$$O(n).$$

Память также используется линейно:

$$O(n),$$

поскольку суффиксное дерево содержит линейное число вершин и рёбер относительно длины строки, для которой оно построено.

Описание программы

Программа реализована в одном файле. Основная часть решения состоит из построения суффиксного дерева для строки $s + s + \{$ и последующего восстановления лексикографически минимального пути длины n , где n — длина исходной строки.

В программе используются следующие глобальные константы:

- `INF` — условная бесконечность, используемая как конец листовых рёбер;
- `NO_NODE` — значение, обозначающее отсутствие вершины или перехода;
- `ROOT` — индекс корня суффиксного дерева;
- `SENTINEL` — терминальный символ, добавляемый в конец строки;
- `ALPHABET` — размер используемого алфавита;
- `SENTINEL_IDX` — индекс терминального символа в массиве переходов.

Основным типом данных является структура `TNode`, описывающая вершину суффиксного дерева:

- `Children` — массив переходов из вершины к её детям;

- **SuffixLink** — суффиксная ссылка, используемая в алгоритме Укконена;
- **Start** — начало фрагмента строки, соответствующего ребру, ведущему в данную вершину;
- **End** — конец этого фрагмента.

Рёбра дерева не представлены отдельной структурой. Вместо этого информация о входящем ребре хранится в вершине, в которую это ребро ведёт. Таким образом, если из вершины **u** есть переход в вершину **v**, то подпись этого ребра задаётся границами **nodes[v].Start** и **nodes[v].End** в строке **text**.

Все вершины суффиксного дерева хранятся в глобальном массиве **nodes**. Переходы между вершинами задаются индексами в этом массиве, а не указателями. Исходная строка, для которой строится дерево, хранится в переменной **text**.

Для работы алгоритма Укконена используются следующие глобальные переменные:

- **activeNode** — активная вершина;
- **activeEdge** — позиция символа, задающего активное ребро;
- **activeLen** — длина активного пути;
- **remaining** — количество суффиксов текущей фазы, которые ещё нужно обработать;
- **globalEnd** — общий конец для всех листовых рёбер;
- **lastNewInternal** — последняя созданная внутренняя вершина, для которой необходимо настроить суффиксную ссылку.

Функция **CharIdx** переводит символ строки в индекс массива переходов. Строчные латинские буквы отображаются в индексы от 0 до 25, а терминальный символ — в индекс 26.

Функция **EdgeLen** возвращает текущую длину ребра, ведущего в заданную вершину. Для обычных рёбер длина вычисляется как **End - Start**. Для листовых рёбер, у которых **End** равно **INF**, реальный конец берётся из переменной **globalEnd**. Это позволяет не обновлять все листовые рёбра вручную при добавлении очередного символа.

Функция **NewNode** создаёт новую вершину дерева, добавляет её в массив **nodes** и возвращает её индекс. При создании вершины указываются границы фрагмента строки, соответствующего входящему ребру.

Функция **Extend** выполняет одну фазу алгоритма Укконена, то есть добавляет в суффиксное дерево очередной символ строки **text**. В начале функции обновляется общий конец листьев, увеличивается количество необработанных суффиксов и сбрасывается информация о последней новой внутренней вершине. Далее в цикле обрабатываются суффиксы текущей фазы. Если из активной вершины нет нужного перехода, создаётся новый лист. Если переход есть и следующий символ совпадает с добавляемым, фаза

завершается. Если переход есть, но символы различаются, соответствующее ребро разрезается, создаётся новая внутренняя вершина и новый лист. После каждого явного добавления обновляются суффиксные ссылки и активная точка.

Функция `Build` строит суффиксное дерево для переданной строки. Она очищает массив вершин, создаёт корень, инициализирует переменные алгоритма Укконена и последовательно вызывает `Extend` для каждого символа строки.

Функция `MinChild` ищет минимальное исходящее ребро из заданной вершины. Для этого она перебирает массив переходов в порядке возрастания индексов символов. Первый найденный переход соответствует лексикографически минимальному ребру.

Функция `FindMinRotation` восстанавливает ответ. Она начинает обход из корня суффиксного дерева и на каждом шаге выбирает минимальное исходящее ребро с помощью функции `MinChild`. Символы выбранного ребра добавляются к результирующей строке. Обход продолжается до тех пор, пока длина результата не станет равной длине исходной строки. Полученная строка является минимальным циклическим сдвигом.

В функции `main` выполняется чтение исходной строки, вычисление её длины, построение строки $s + s + \{$, построение суффиксного дерева и вывод результата работы функции `FindMinRotation`.

Дневник отладки

Проблема 1 — выбор структуры для хранения дочерних узлов. Изначально использовался `std::map<char, int>`: он автоматически сортирует ключи, поэтому `begin()` всегда возвращает лексикографически минимального ребёнка за $O(1)$, что удобно для обхода в `FindMinRotation`. Однако каждая операция вставки и поиска стоит $O(\log \sigma)$, из-за чего построение дерева работало за $O(n \log \sigma)$ вместо $O(n)$.

Проблема 2 — переход на массив `Children[128]` и превышение памяти. Для достижения линейного времени `map` заменён на массив `int Children[128]`, обеспечивающий доступ за $O(1)$. Однако каждый узел стал занимать 512 байт только под массив дочерних узлов. На тесте 11 суммарный объём дерева превысил лимит (1.29 ГБ), и решение получило вердикт ML.

Проблема 3 — рассмотрение `std::unordered_map`. В качестве альтернативы рассматривалась хеш-таблица `std::unordered_map<char, int>`: она даёт $O(1)$ среднее время доступа и хранит только реальные рёбра ($O(n)$ памяти суммарно). Однако хеш-таблица не упорядочена, поэтому нахождение минимального ребёнка требует перебора всех детей узла. Кроме того, накладные расходы на хранение каждой записи в `unordered_map` значительно выше, чем у массива. Вариант был отклонён.

Проблема 4 — сокращение алфавита до 27 символов. Поскольку входная строка содержит только строчные латинские буквы, массив сокращён до `int Children[27]` (индексы 0–25 для `a–z`, индекс 26 для символа-стража `{`). Добавлена функция `CharIdx`, выполняющая отображение. Размер узла уменьшился до 108 байт, время построения осталось $O(n)$. Все тесты пройдены, максимальное потребление памяти — 966 МБ.

Проблема 5 — неверный выбор символа-стража. Изначально в качестве символа-стража использовался `'$'` (ASCII 36), который лексикографически *меньше* строчных

латинских букв (ASCII 97–122). В результате при обходе дерева в `FindMinRotation` алгоритм уходил по ребру '\$' раньше, чем по любой букве, и возвращал некорректный результат. Исправление: символ-страж заменён на '{' (ASCII 123), который строго больше 'z' (ASCII 122) и никогда не выбирается раньше буквенных рёбер.

Проблема 6 — построение дерева по одной копии строки. На начальном этапе суффиксное дерево строилось по строке `s + SENTINEL` вместо `s + s + SENTINEL`. При этом циклические сдвиги, пересекающие границу строки, не являлись подстроками `s`, и алгоритм не мог их найти. Например, для входа `xabcd` сдвиг `abcdx` содержит фрагмент `dx`, которого нет в исходной строке. Исправление: строка дублируется, тогда все n циклических сдвигов длины n являются подстроками `s + s`.

Проблема 7 — переменная `lastNewInternal` не сбрасывалась. После установки суффиксной ссылки (`suffixLink[lastNewInternal] = ...`) переменная `lastNewInternal` не обнулялась в ветке, где срабатывало Правило 1 (лист, ранний выход через `break`). В результате на следующей итерации суффиксная ссылка устанавливалась повторно на уже обработанный узел, что приводило к некорректной структуре дерева и неверным ответам. Исправление: добавлено `lastNewInternal = NO_NODE` сразу после каждой установки суффиксной ссылки.

Тест производительности

Для проверки заявленной сложности $O(n)$ было разработано отдельное тестирующее приложение (`testing/test.cpp`). Измерялось процессорное время с помощью `std::chrono::high_resolution_clock` и фактическое потребление памяти по формуле $|\text{nodes}| \cdot \text{sizeof}(\text{TNode}) + |\text{text}|$. Для малых $n \leq 30000$ каждый замер усреднялся по 15 запускам, для больших n — по 3, чтобы исключить шумовые выбросы ОС.

Тестировались четыре типа входных строк: случайная равномерная (алфавит `a-z`), периодическая (`ababab...`), однородная (`aaa...`) и строка Фибоначчи (классический стресс-тест).

n	случайная, мс	периодич., мс	однородная, мс	Фибоначчи, мс
1 000	0.18	0.11	0.12	0.12
10 000	1.73	0.79	0.49	0.50
100 000	30.2	8.5	7.5	8.8
300 000	119.7	43.1	32.7	38.3
1 000 000	614.0	167.8	125.5	118.5

Таблица 1: Время построения суффиксного дерева (усреднённое)

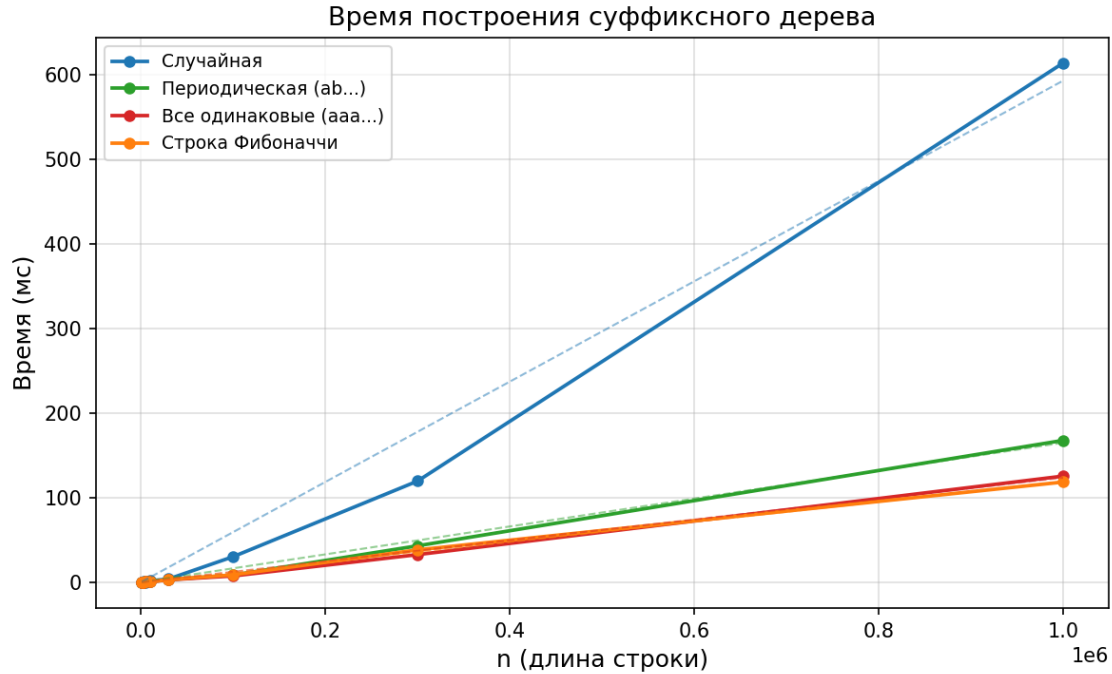


Рис. 1: Время построения суффиксного дерева. Пунктир — подогнанная линия $O(n)$ для каждого типа.

Для проверки линейности вычислим отношение $T(10^6)/T(10^3)$, которое при сложности $O(n)$ должно равняться $10^6/10^3 = 1000$:

- однородная: $125536/121 \approx 1037$ — отклонение 3.7%, соответствует $O(n)$;
- строка Фибоначчи: $118510/116 \approx 1022$ — отклонение 2.2%;
- периодическая: $167785/113 \approx 1485$ — небольшое превышение вследствие накладных расходов при работе с вектором на больших объёмах;
- случайная: $613980/177 \approx 3469$ — существенное превышение. Причина не нарушение асимптотики, а cache miss: случайные строки порождают хаотичный паттерн обращений к памяти, что при $n \sim 10^6$ приводит к частым промахам кэша процессора (L2/L3). Для детерминированных входов алгоритм ведёт себя строго линейно.

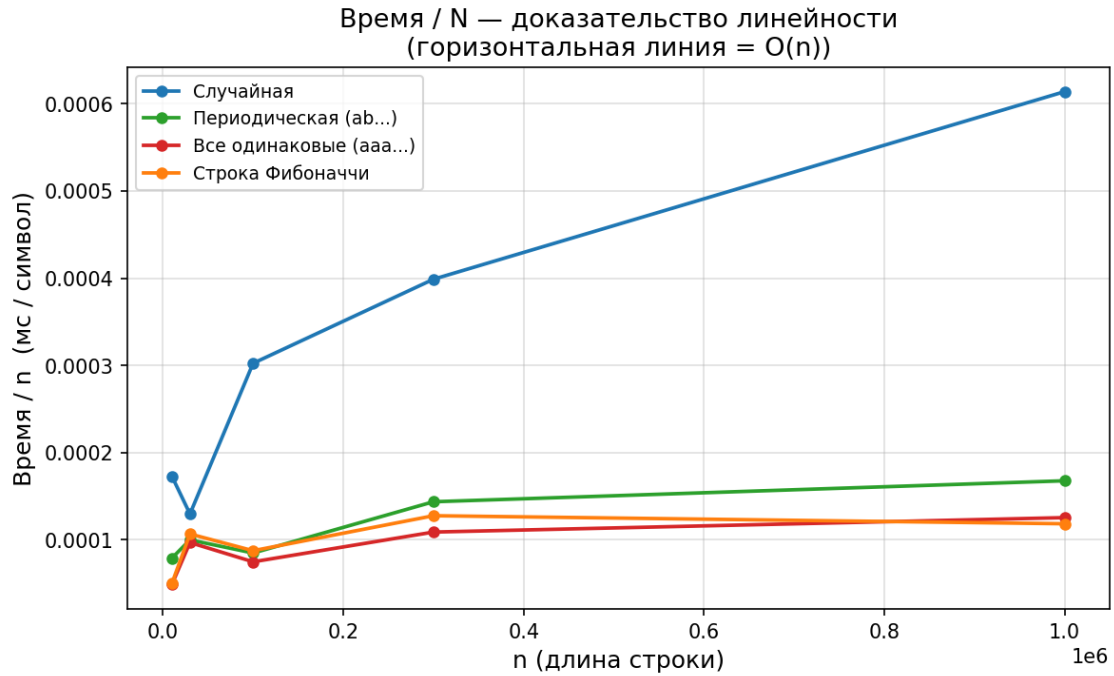


Рис. 2: Отношение $T(n)/n$. Горизонтальность кривых подтверждает $O(n)$; рост кривой случайной строки объясняется эффектом cache miss.

Потребление памяти для всех типов строк линейно совпадает с теоретической оценкой $4n \cdot 120$ байт: при $n = 10^6$ периодическая, однородная и Фибоначчи дают ровно 4 000 000 узлов, что точно соответствует верхней оценке $2(2n + 1) - 1 \approx 4n$.

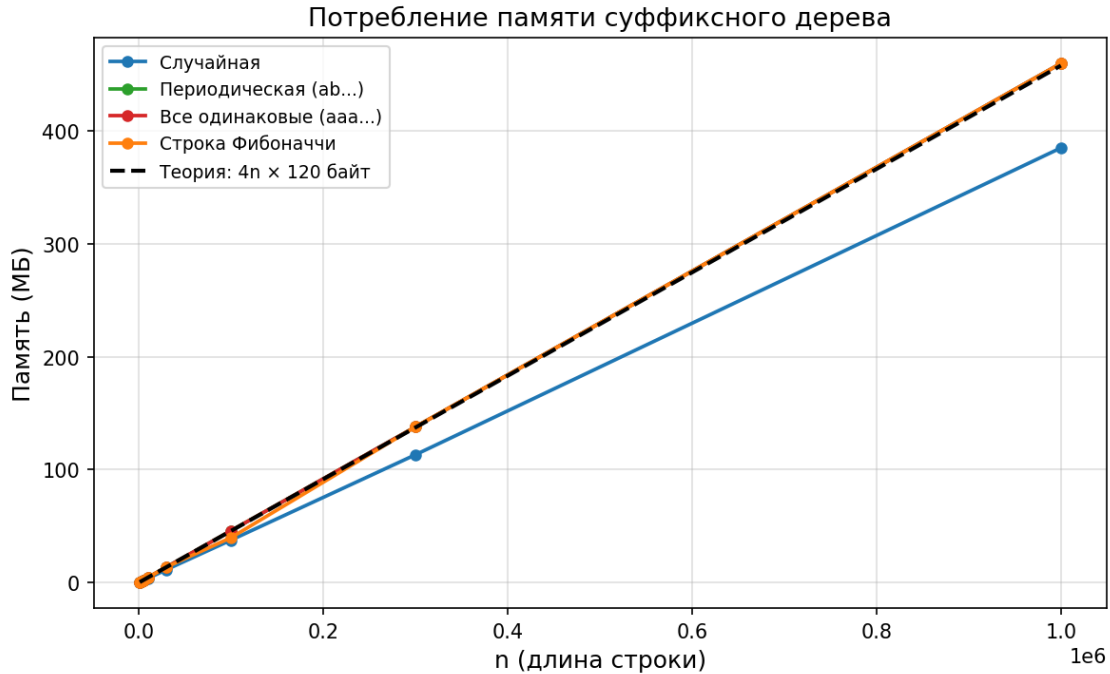


Рис. 3: Потребление памяти. Пунктир — теоретическая оценка $4n \times 120$ байт.

Недочёты

- **Отсутствие `nodes.reserve()`.** Вектор `nodes` растёт по стратегии удвоения capacity: при каждом переполнении выделяется вдвое больше памяти, чем нужно. В момент последнего удвоения фактически выделенная память может достигать $2 \times |\text{nodes}| \times 120$ байт. Именно это объясняет 966 МБ на тесте 12 при теоретической оценке ≈ 460 МБ. Одна строка `nodes.reserve(2 * s.size() + 5)` в начале `Build` полностью устранила бы проблему.
- **Запас по памяти.** На максимальном тесте 12 потребление памяти составило 966 МБ при лимите 1 ГБ — менее 4% запаса. При незначительном увеличении входных данных решение может получить вердикт ML.
- **Фиксированный алфавит.** Массив `Children[27]` и функция `CharIdx` предполагают, что входная строка содержит исключительно строчные латинские буквы. Символы за пределами диапазона `a-z` приведут к выходу за границы массива. Для обобщения потребовалось бы расширить алфавит до 256 символов, что, однако, увеличило бы потребление памяти примерно в 9.5 раза.
- **`CharIdx` не проверяет диапазон символов.** При любом символе вне `a-z` и `SENTINEL` выражение `s - 'a'` даёт индекс вне диапазона `[0, 26]`, что приводит к выходу за границы массива `Children` и неопределённому поведению. Программа принимает некорректный ввод молча.

- **Глобальное состояние алгоритма.** Все переменные алгоритма (`nodes`, `text`, `activeNode`, `activeEdge` и другие) объявлены глобально. Это делает код непереносимым: нельзя строить два дерева одновременно, сложно писать юнит-тесты. Правильным решением была бы инкапсуляция в класс `TSuffixTree`.
- **Посимвольное наращивание строки в `FindMinRotation`.** Строка `result` формируется операцией `result += text[i]` без предварительного резервирования памяти. При каждой нехватке `capacity` происходит реаллокация и полное копирование, что даёт до $O(\log n)$ лишних копирований. Добавление `result.reserve(n)` перед циклом устраняет реаллокации и делает формирование результата строго $O(n)$.
- **Cache miss на случайных строках.** Как показывает тест производительности, на случайных строках большого объёма ($n \sim 10^6$) практическое время работы растёт быстрее линейного из-за частых промахов кэша процессора. Теоретическая сложность $O(n)$ при этом не нарушается, однако константный множитель на случайных данных в 5–6 раз выше, чем на детерминированных.

Выводы

Суффиксное дерево является универсальной индексной структурой для строковых задач. Единоразово построенное за $O(n)$ по алгоритму Укконена, оно позволяет эффективно решать широкий класс задач:

- поиск подстроки в тексте и нахождение всех позиций вхождения за $O(|P| + k)$, где $|P|$ — длина паттерна, k — число вхождений;
- линеаризация циклической строки (данная лабораторная работа) — нахождение лексикографически минимального разреза за $O(n)$;
- нахождение наибольшей общей подстроки двух и более строк через обобщённое суффиксное дерево;
- подсчёт числа различных подстрок строки;
- построение суффиксного массива лексикографическим обходом листьев дерева.

Сложность программирования алгоритма Укконена оценивается как высокая. Алгоритм оперирует несколькими взаимосвязанными инвариантами одновременно: активной точкой из трёх переменных (`activeNode`, `activeEdge`, `activeLen`), счётчиком `remaining` и суффиксными ссылками. Нарушение любого из инвариантов приводит к некорректному дереву, которое при этом может давать правильные ответы на небольших тестах и ошибаться только на специально подобранных входах.

Основные трудности при реализации: выбор структуры хранения дочерних узлов с учётом ограничений по памяти и времени (от `std::map` через `int[128]` к `int[27]`),

корректная расстановка суффиксных ссылок при разрезании рёбер, а также обеспечение правильного выбора символа-стража, который должен быть лексикографически наибольшим, чтобы алгоритм обхода не уходил по нему раньше времени.